Virtual-Interface-Based RTL Verification and FPGA Deployment of a 16-bit ALU

Adrian Humberto Valle Araujo
Verificación UVM
Universidad Politécnica de Yucatán
Mérida, México
2309218@upy.edu.mx

Abstract*—This paper documents the final project for a hardware verification course, in which a 16-bit Arithmetic Logic Unit (ALU) was designed in SystemVerilog and verified through a layered, UVM-inspired testbench architecture rather than through direct, hand-written stimulus. The Design Under Test (DUT) is decoupled from its stimulus through a Virtual Interface (VIF) with a clocking block, and a separate `test` module drives randomized inputs into the DUT purely through that interface, while a top-level `tb` module is responsible only for clock generation and module instantiation. Compilation, elaboration, and simulation were performed entirely from a Linux terminal using the Xilinx Vivado simulator invoked through a Makefile (`make compile`, `make elaborate`, `make sim`), without opening the full Vivado GUI, reflecting the terminal-driven workflow emphasized throughout the course as representative of industry practice. The verified ALU was then adapted into a top-level wrapper, `alu_top`, and deployed on a Terasic DE2-115 board (Cyclone IV, EP4CE115F29C7), using the board's switches and keys to load operands and control bits and its LEDs to display the result and status flags. Static timing analysis (STA) was performed against a Synopsys Design Constraints (SDC) file constrained to the board's actual 50 MHz oscillator (20 ns period, `CLOCK_50`), which yielded positive setup slack across all three PVT corners evaluated, in contrast to the arbitrary, higher-frequency constraints that are more commonly assumed and that would have produced negative slack for this same design.

***Index Terms*—SystemVerilog, virtual interface, UVM-style verification, ALU, Vivado simulator, static timing analysis, FPGA, Cyclone IV**

I. Introduction

Directed testbenches, in which a fixed sequence of stimulus values is applied to a Design Under Test (DUT) and the corresponding outputs are checked by inspection, are adequate for small combinational blocks but do not scale to the kind of coverage-driven, randomized verification used in industry. In the verification course this project was developed for, the emphasis was placed on separating the DUT, the stimulus, and the bench itself into independent, reusable components, following the layering philosophy that underlies the Universal Verification Methodology (UVM), even though the full UVM class library was not used directly. Instead, this separation was achieved with three plain SystemVerilog constructs: a `virtual interface` (VIF) that exposes the DUT's ports together with a clocking block for synchronous, race-free driving and sampling; a `test` module that owns all stimulus-generation and monitoring tasks and communicates with the DUT exclusively through the VIF; and a top-level `tb` module whose only responsibilities are clock generation and instantiating both the DUT and the `test` module against the same interface handle.

A second emphasis of the course was that this entire flow should be executable from the command line, the way it is done in industry, rather than through a GUI-driven simulation. All compilation, elaboration, and simulation steps in this project were therefore run from a Linux terminal using the Xilinx Vivado simulator (`xvlog`/`xelab`/`xsim`) wrapped in a Makefile, invoked with `make compile`, `make elaborate`, and `make sim`, without ever opening the Vivado IDE.

Beyond RTL-level verification, the same ALU design was adapted for physical deployment on a Terasic DE2-115 development board (Altera/ Intel Cyclone IV, part number EP4CE115F29C7), and its timing was characterized with static timing analysis (STA) under the board's real clock source. This paper describes the ALU architecture, the VIF-based verification environment, the terminal-driven simulation flow, the FPGA adaptation, and the resulting functional and timing results.

II. Background

A. Virtual Interfaces and Clocking Blocks

A SystemVerilog `interface` bundles a set of related signals into a single handle that can be passed between modules, avoiding long, error-prone port lists. When an interface instance is passed into a class or a separate stimulus module as a handle rather than being hard-wired at elaboration time, it is commonly referred to as a *virtual interface* (VIF); this is the mechanism UVM itself uses to connect class-based stimulus components to a DUT. A `clocking block` declared inside the interface further specifies input and output skew relative to a reference clock edge, so that driving a signal through `vif.cb.signal <= value` and sampling it through `vif.cb.signal` are both race-free with respect to the DUT's own clocked always blocks. This project uses both mechanisms together: a single `vif_if` interface with a clocking block synchronized to `clk_i` is shared between the DUT instantiation and the `test` module.

B. Command-Line Simulation with Vivado

The Vivado Simulator can be driven entirely from the command line through three underlying tools: `xvlog` to compile SystemVerilog sources, `xelab` to elaborate the design and build a simulation snapshot, and `xsim` to run that snapshot. Wrapping these three steps in a Makefile with targets such as `compile`, `elaborate`, and `sim` allows the entire verification flow to be reproduced with three short commands and no graphical interface, which is the workflow followed throughout this project.

C. ALU Architecture

The ALU implements the classic 16-bit, six-control-bit architecture popularized by the Hack computer design: two 16-bit operands (`x_i`, `y_i`) can each be independently zeroed (`zx_i`/`zy_i`) and negated (`nx_i`/ `ny_i`) before being combined, either by addition or by a bitwise AND selected by `f_i`, and the combined result can optionally be bitwise-negated by `no_i`. Concatenating these six control bits as {`zx_i, nx_i, zy_i, ny_i, f_i, no_i`} produces a 6-bit instruction word that is decoded by a single `unique case` statement into one of seventeen named operations (Table I); the DUT also registers three status flags — zero, negative, and positive — on the following clock edge.

D. Setup Slack and Static Timing Analysis

Static timing analysis reports the setup slack of a design as the difference between the required arrival time of a signal, set by the clock period and any input/output delay constraints, and its actual arrival time after propagating through combinational logic. Negative slack indicates a timing violation for the specified clock period; positive slack indicates margin. Because slack is measured relative to whatever clock period is declared in the constraints file, the same synthesized netlist can show negative slack under an aggressively fast constraint and positive slack under a constraint that matches the clock actually available on the target board.

III. Methodology

A. Development Tools and Workflow

RTL design was performed in Visual Studio Code on Linux. Functional simulation was performed with the Vivado Simulator, invoked purely from the terminal through a Makefile: `make compile` calls `xvlog` to compile every `.sv` source file, `make elaborate` calls `xelab` to build the simulation snapshot, and `make sim` calls `xsim` to run it and open the resulting waveform. The full Vivado GUI was never opened; only the underlying simulator binaries were used, in line with the course's emphasis that this is how verification is done in industry. Synthesis, pin assignment, and static timing analysis for the FPGA target were performed in Quartus Prime, targeting the Cyclone IV device on the DE2-115 board.

B. ALU Design Under Test

The DUT, `alu`, is parameterized by `DataWidth` (16 in this project) and implements the operation table of Table I inside a single `always_comb` block using a `unique case` statement on the concatenated 6-bit instruction word, followed by a synchronous `always_ff` block that registers the zero, negative, and positive status flags on the next rising edge of `clk_i` (Listing 1).

TABLE I: 16-bit ALU Operation Table

Control {zx,nx,zy,ny,f,no}	**Name**	**Result**
101010	ZERO	0
111111	ONE	1
111010	MONE	-1
001100	X	x
110000	Y	y
001101	NOTX	$\bar{x}$
110001	NOTY	$\bar{y}$
001111	NEGX	$-x$
110011	NEGY	$-y$
011111	XP1	$x+1$
110111	YP1	$y+1$
001110	XM1	$x-1$
110010	YM1	$y-1$
000010	ADD	$x+y$
010011	SUBXY	$x-y$
000111	SUBYX	$y-x$
000000	AND	$x \,\&\, y$
010101	OR	$x \mid y$

Listing 1: alu.sv — 16-bit ALU DUT (decoder and flag register).

```
assign inst = {zx_i, nx_i, zy_i, ny_i, f_i, no_i};

always_comb begin
  unique case (inst)
    ALU_ZERO:  out_o = '0;
    ALU_ONE:   out_o = 1;
    ALU_MONE:  out_o = '1;
    ALU_X:     out_o = x_i;
    ALU_Y:     out_o = y_i;
    ALU_NOTX:  out_o = ~x_i;
    ALU_NOTY:  out_o = ~y_i;
    ALU_NEGX:  out_o = -x_i;
    ALU_NEGY:  out_o = -y_i;
    ALU_XP1:   out_o = x_i + 1;
    ALU_YP1:   out_o = y_i + 1;
    ALU_XM1:   out_o = x_i - 1;
    ALU_YM1:   out_o = y_i - 1;
    ALU_ADD:   out_o = x_i + y_i;
    ALU_SUBXY: out_o = x_i - y_i;
    ALU_SUBYX: out_o = y_i - x_i;
    ALU_AND:   out_o = x_i & y_i;
    ALU_OR:    out_o = x_i | y_i;
    default:   out_o = '0;
  endcase
end

always_ff @(posedge clk_i or posedge rst_i) begin
  if (!rst_i) begin
    zr_o  <= 0;  ng_o <= 0;  pos_o <= 0;
  end else begin
    zr_o  <= (out_o == '0);
    ng_o  <= out_o[DataWidth-1];
    pos_o <= !zr_o && !ng_o;
  end
end
```

C. Virtual Interface

The interface `vif_if` bundles every DUT port and declares a clocking block `cb` synchronized to `clk_i`, with the inputs of the DUT (`rst_i`, `x_i`, `y_i`, and the six control bits) exposed as clocking-block outputs so the `test` module can drive them race-free, and the DUT's outputs exposed as clocking-block inputs for race-free sampling (Listing 2).

Listing 2: vif_if.sv — shared virtual interface with clocking block.

```
interface vif_if (input logic clk_i);
  logic rst_i;
  logic [DataWidth-1:0] x_i, y_i;
  logic zx_i, nx_i, zy_i, ny_i, f_i, no_i;
  logic [DataWidth-1:0] out_o;
  logic zr_o, ng_o, pos_o;

  clocking cb @(posedge clk_i);
    default input #1ns output #1ns;
    output rst_i, x_i, y_i, zx_i, nx_i, zy_i, ny_i, f_i,
         no_i;
    input  out_o, zr_o, ng_o, pos_o;
  endclocking
endinterface : vif_if
```

D. Testbench and Randomized Test

The top-level `tb` module (Listing 3) is intentionally minimal: it generates a 100 MHz simulation clock, instantiates the `vif_if` interface, and instantiates both the `alu` DUT and the `test` module against the same interface handle. It contains no stimulus of its own.

Listing 3: tb.sv — clock generation and instantiation only.

```
logic clk_i = 0;
always #(MainClkPeriod / 2) clk_i = ~clk_i;

vif_if vif (clk_i);
test   top_test (vif);

alu #(.DataWidth(WIDTH)) dut (
  .clk_i(vif.clk_i), .rst_i(vif.rst_i),
  .x_i(vif.x_i), .y_i(vif.y_i),
  .zx_i(vif.zx_i), .nx_i(vif.nx_i),
  .zy_i(vif.zy_i), .ny_i(vif.ny_i),
  .f_i(vif.f_i), .no_i(vif.no_i),
  .out_o(vif.out_o), .zr_o(vif.zr_o),
  .ng_o(vif.ng_o), .pos_o(vif.pos_o)
);
```

All stimulus, reset, and monitoring logic instead lives in the `test` module, which drives the DUT only through `vif.cb` and runs a reset task, a randomized-stimulus task, and a monitor task concurrently through a `fork...join_any` / `join_none` structure (Listing 4). One hundred randomized `x_i`/`y_i` pairs are generated with `$urandom_range` while the control bits are held at a fixed ADD configuration for this run, and every sample is logged from the monitor task with `$display`.

Listing 4: test.sv (excerpt) — reset, randomized stimulus, and monitor tasks.

```
initial begin
  fork
    monitor_output();
  join_none
  reset();
  fork
    send_data_port();
  join_any
  #(20ns);
  $finish;
end

task automatic reset();
  vif.x_i = 16'b0; vif.y_i = 16'b0; vif.rst_i = 1'b1;
  repeat (5) @(vif.cb);
  vif.rst_i = 1'b0;
endtask

task automatic send_data_port();
  for (int i = 0; i < 100; i++) begin
    @(vif.cb);
    vif.cb.x_i  <= $urandom_range(0, 15);
    vif.cb.y_i  <= $urandom_range(0, 15);
    vif.cb.zx_i <= '0; vif.cb.nx_i <= '1;
    vif.cb.zy_i <= '0; vif.cb.ny_i <= '0;
    vif.cb.f_i  <= '1; vif.cb.no_i <= '1;
  end
endtask
```

E. FPGA Adaptation

To deploy the verified DUT on hardware, the original `alu` module was wrapped in a top-level module, `alu_top`, adapted to the DE2-115's available I/O rather than to the VIF used in simulation (Listing 5). The board's 18 slide switches `SW[17:0]` supply a 16-bit data bus and a 2-bit mode selector; pressing `KEY[0]` loads that data, on the next rising edge of the 50 MHz board clock `CLOCK_50`, into either the `x` register, the `y` register, or the 6-bit control/ flag register, depending on the mode selector, while `KEY[3]` acts as an active-low asynchronous reset. The ALU result is displayed continuously on `LEDR[15:0]`, the active mode on `LEDR[17:16]`, and the zero/negative/positive flags on `LEDG[2:0]`.

Listing 5: alu_top.sv (excerpt) — FPGA-facing wrapper for the DE2-115.

```
always_ff @(posedge CLOCK_50 or negedge rst_n) begin
  if (!rst_n) begin
    reg_x <= '0; reg_y <= '0; reg_flags <= '0;
  end else if (key0_sync0) begin
    unique case (mode_sel)
      2'b00:   reg_x     <= data_bus;
      2'b01:   reg_y     <= data_bus;
      2'b10:   reg_flags <= data_bus[5:0];
      default: ;
    endcase
  end
end

alu #(.DataWidth(DataWidth)) u_alu (
  .x_i(reg_x), .y_i(reg_y),
  .zx_i(reg_flags[5]), .nx_i(reg_flags[4]),
  .zy_i(reg_flags[3]), .ny_i(reg_flags[2]),
  .f_i(reg_flags[1]),  .no_i(reg_flags[0]),
  .out_o(alu_out), .zr_o(zr), .ng_o(ng), .pos_o(pos)
);

assign LEDR[15:0]  = alu_out;
assign LEDR[17:16] = mode_sel;
assign LEDG = {pos, ng, zr};
```

F. Timing Constraints

The SDC file applied for STA constrains the design to `CLOCK_50`, the DE2-115's actual 50 MHz onboard oscillator, declared with a 20 ns period rather than an arbitrary

faster target, together with a 2 ns input and output delay on the switch, key, and LED ports, and false-path exceptions on `SW` and `KEY` since these are asynchronous, user-driven inputs with no meaningful setup/hold relationship to the clock (Listing 6). Constraining the design to the clock frequency that the board actually supplies, instead of the higher, more commonly assumed frequency used as a default target in earlier timing exercises, is what allowed this design to close timing with positive slack; the same netlist analyzed against a faster, arbitrarily chosen constraint would be expected to show negative slack, as is typically the case for ALU decode logic of this depth.

Listing 6: Synopsys Design Constraints (SDC) file for the FPGA target.

```
create_clock -period 20.000 -name CLOCK_50 [get_ports {
     CLOCK_50}]

set_input_delay  -clock CLOCK_50 2.0 [get_ports {SW[*]}]
set_input_delay  -clock CLOCK_50 2.0 [get_ports {KEY[*]}]
set_output_delay -clock CLOCK_50 2.0 [get_ports {LEDR[*]}]
set_output_delay -clock CLOCK_50 2.0 [get_ports {LEDG[*]}]

set_false_path -from [get_ports {SW[*]}]
set_false_path -from [get_ports {KEY[*]}]
```

Static timing analysis was run in Quartus Prime under three corner models: slow 1200 mV 85 °C, slow 1200 mV 0 °C, and fast 1200 mV 0 °C, and the worst setup slack reported under each corner was recorded.

IV. Results

A. Functional Simulation

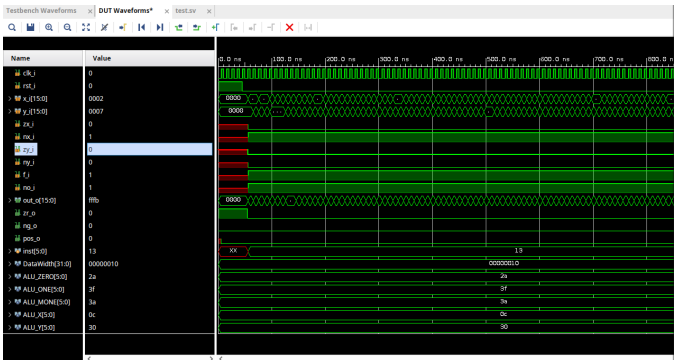


Fig. 1: Waveforms of Vivado simulation

Fig. 1 shows the `xsim` waveform produced by `make sim`, captured after reset deassertion. The randomized `x_i`/`y_i` pairs generated by `send_data_port()` propagate through the ALU decoder combinationally, and `out_o` together with the registered `zr_o`/`ng_o`/`pos_o` flags update as expected on each sampled clock edge, consistent with the ADD configuration driven for this run.

During the first five clock cycles, several control signals in the waveform appear as indeterminate (`X`). This is a direct consequence of the `test` module's sequential structure: the `reset()` task, which only drives `x_i`, `y_i`, and `rst_i`, executes to completion *before* `send_data_port()` is launched, so the control bits (`zx_i`, `nx_i`, `zy_i`, `ny_i`, `f_i`, `no_i`) are never assigned during those five reset cycles and simply retain their uninitialized value. This can be resolved either by also assigning explicit values to the control bits inside `reset()`, or by restructuring the `initial` block so that `reset()` and `send_data_port()` run concurrently inside the same `fork...join` block instead of sequentially, so that the control bits are driven to a known value from the first cycle onward.

B. Static Timing Analysis

Table II reports the worst setup slack obtained under each of the three STA corners once the design was constrained to the board's actual 50 MHz clock (20 ns period). All three corners report positive slack, in contrast to the negative slack that this same combinational depth would be expected to produce under a faster, arbitrarily chosen clock constraint.

TABLE II: Worst Setup Slack for the 16-bit ALU at 50 MHz (ns)

Design	Slow 85°C	Slow 0°C	Fast 0°C
alu_top (20 ns / 50 MHz)	1.096	2.697	9.062

C. FPGA Verification

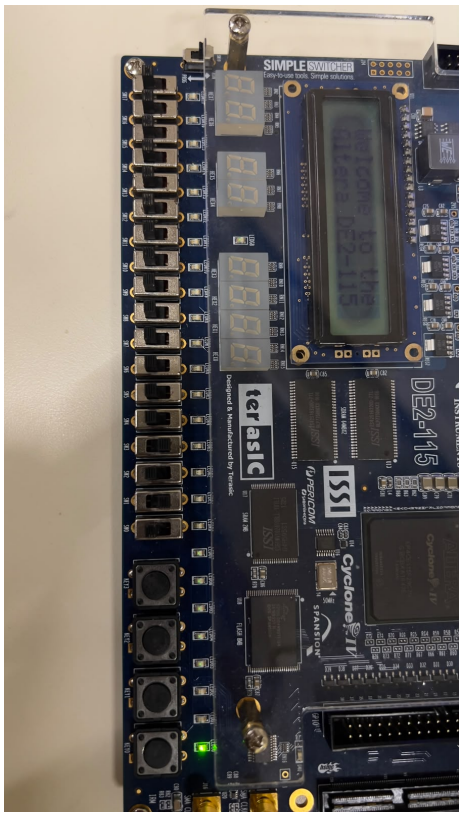


Fig. 2: SEL = 00 (X) — DATA = 0x000A

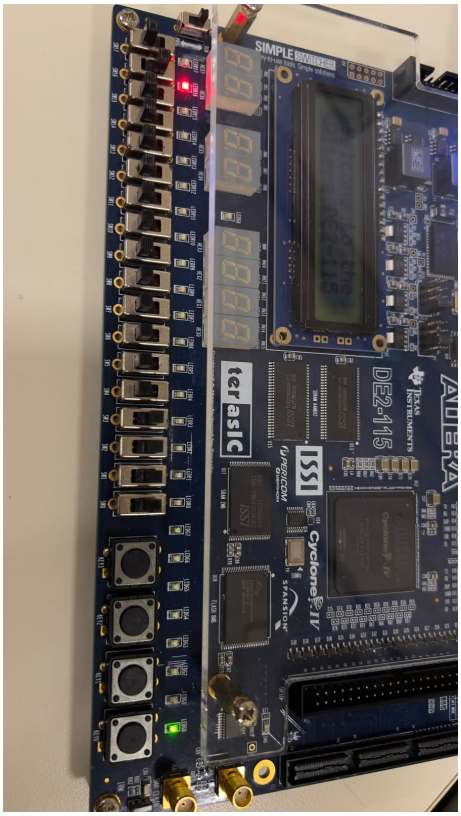


Fig. 3: SEL = 01 (Y) — DATA = 0x000F

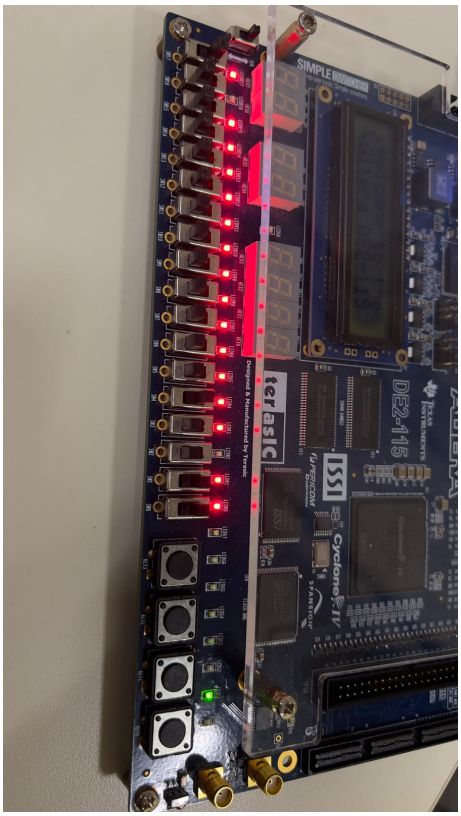


Fig. 4: SEL = 10 (FLAGS) — DATA = 0x0014

Figs. 2–**??** show the `alu_top` design running on the DE2-115 board: the board is powered on and the default welcome screen is shown, the operand and mode switches are set, `KEY[0]` is pressed to load the operands and control bits into their respective registers, and the resulting output is displayed across the red LED bank (`LEDR`), confirming correct operation of the same DUT that was verified through the VIF-based simulation environment.

V. Discussion

The layered VIF/test/tb structure kept the DUT, the stimulus, and the bench fully decoupled: the `alu` module was never modified to support verification, the `test` module never referenced the DUT directly, and the same `vif_if` handle could, in principle, be reused against a different implementation of the same port list without any change to the stimulus code. Running this entire flow from three Makefile targets calling the underlying Vivado simulator binaries, without opening the Vivado GUI, reproduced the terminal-based workflow the course identified as representative of industry practice.

On the timing side, constraining the FPGA implementation to the board's real 50 MHz clock rather than to a faster, arbitrarily chosen target was the deciding factor in obtaining positive setup slack across all three PVT corners. This illustrates a more general point: setup slack is only meaningful relative to the clock period actually declared in the constraints, and a design that appears to fail timing under an assumed frequency may comfortably meet timing once it is constrained to the frequency it will actually run at on the target hardware.

Conclusion

A 16-bit ALU was verified through a VIF-based, UVM-inspired testbench in which a dedicated `test` module drove one hundred randomized stimulus vectors into the DUT exclusively through a clocking-block interface, while a minimal `tb` module handled only clock generation and instantiation. The full compile/elaborate/simulate flow was executed from a Linux terminal using the Vivado simulator through a Makefile, without the Vivado GUI. The same DUT was subsequently wrapped for deployment on a DE2-115 (Cyclone IV) board and verified functionally on hardware, and static timing analysis showed that constraining the design to the board's actual 50 MHz clock (20 ns period), rather than an arbitrarily faster target, was what allowed positive setup slack to be achieved across all three PVT corners evaluated.

References

[1] IEEE, "IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language," IEEE Std 1800-2017, IEEE, New York, NY, USA, 2018.

[2] Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User's Guide," Accellera Systems Initiative, 2015.

[3] Xilinx (AMD), "Vivado Design Suite User Guide: Logic Simulation," UG900, Xilinx Inc., 2022.

[4] Intel Corporation, "Quartus Prime Standard Edition Handbook," vol. 1: Design and Compilation, Intel Corporation, 2020.